Agent × Robot: From Harness to Self-Evolving Physical Agents

A reading of four source materials, a 2026 survey of 23 topic lines, and ten insights

awesome_agentic_robot · asimfish

2026-09-06

Contents

Executive summary

This report answers three questions. What happened on the Agent × Robot line in 2026. What each of five source materials (a long-form essay on Xiaohongshu, a 25-slide Code-as-Policy deck, a two-page retrieval map with 23 topic lines, two Lil'Log posts by Lilian Weng, and a WeChat essay on Robot RSI by 具身纪元) gets right and what it leaves out. And, looking along those lines, which claims are backed by published evidence and which are still predictions.

Six conclusions.

1. **The object being optimised is moving up the stack.** In 2022 Code as Policies asked an LLM to write one policy program. In 2026 SkillOpt, ASPIRE, RHO and ENPIRE ask a coding agent to optimise a skill document, a multi-file policy repository, a training recipe, or an entire harness. The deck's unifying formula $z_{t+1} = A(z_t, \tau_t, r_t, \log_t)$ captures the shift: the thing being learned, z, has become an external, trainable artifact rather than an action.
2. **"Frozen VLA plus learning around it"became the default recipe of 2026.** Harness VLA, BATON, AGM, HyMeS, Zetta and SHAPER leave the VLA weights untouched and instead learn the VLA's operating range, memory-management rules, recovery skills and runtime critics outside it. The reason is practical: weight updates are expensive, slow and hard to verify. The ceiling is equally clear: LWD, Q-Planning, TEMPO and Temporal GRPO show that real-world feedback eventually has to be written back into policy weights.
3. **The verifier is the new bottleneck and a new scaling axis.** PhyAgentOS's SessionVerifier, Thea's Evaluation as Exit Codes, AGM's rule that a progress pointer advances only on physical evidence, LLM-as-a-Verifier's treatment of verification as a scalable dimension, and the position paper arguing that success rate cannot certify physical reasoning all point the same way: whether a system can self-evolve depends on whether it can reliably decide "did this step succeed?"
4. **Harness turned from a software term into a robot-middleware problem.** A software harness intervenes at tool-call boundaries; a Physical AI harness has to intervene in control, compute and communication at once (Harness Engineering for Physical AI) and has to know the model's worst-case latency, each skill's deadline and the fallback when the network drops (the essay's "real-time-aware harness"). This is operating-system and real-time-systems design, not prompt engineering.
5. **Fleets are the only route to experience at scale, and the returns are sublinear.** LWD drives a single generalist VLA to 95% with 16 dual-arm robots; ENPIRE cuts convergence from about 5 hours to about 2 hours with 8 stations. Eight times the robots buys two to three times the speed; what scales is hypothesis throughput, not rollout count.
6. **Robot RSI is the frame that ties these together.** The 具身纪元 essay's two axes (what is improved: deployment-time self-evolution / training-time self-iteration / self-evaluation / auto research × how far a human stays in the loop) put ENPIRE, ASPIRE, RoboHarness, RoboClaw, PRIMO R1, VERITAS, Eureka and DrEureka in one

table with the software-side Reflexion, STaR, Let's Verify, Meta-Rewarding and The AI Scientist; a frontier LLM is more likely to become the cognitive hub of the robot research loop than the robot's end-effector controller.

Reading paths: for conclusions only, read Sections 7 and 8; to check the papers behind each topic line, read Section 4 and the appendix; for the interpretation of each source, read Sections 2, 3, 5 and 6. The repository `README.md` lists 208 papers by topic line and `data/papers.csv` is the machine-readable version.

1. Materials, method and scope

1.1 Four inputs

Material	Form	Central claim
"After Harness, what is the next stop for Agent+Robot?" by 具身 RL 日记 (Embodied RL Diary), Xiaohongshu, 39 image cards	Opinion essay	Harness solves "how an agent reliably uses existing capabilities"; the next stops are Memory, Reflection & Evolution, two time scales, Agent+VLA+RL, Real → Sim → Learn → Real, and Multi-Robot; the endgame is System Scaling plus Experience Scaling
Code-as-Policy deck (25 slides, PPTX)	Paper-reading deck	From Code as Policies to SkillOpt, CaP-X, ASPIRE and ENPIRE; self-evolution means optimising an external artifact z
"Agent + Robot paper retrieval map", two-page table	Retrieval framework	23 topic lines, each with search keywords and representative works
Lil'Log: "LLM Powered Autonomous Agents" (June 2023) and "Harness Engineering for Self-Improvement" (July 2026)	Software-side surveys	Agent = Planning + Memory + Tool use; three harness patterns, the optimisation ladder, seven open challenges for recursive self-improvement
"GPT-6 未必能当好机器人的大脑，却可能帮王兴兴加速 RobotRSI" by Marilyn Liu, WeChat account 具身纪元 (with a condensed Xiaohongshu companion note, "GPT-6 Astra 开启 Robot RSI 时代")	Opinion essay	Two axes of RSI (what is improved × human involvement); the LLM is more likely to become the cognitive hub of Robot RSI first; Robot RSI lacks a repeatable, scalable virtual world

1.2 Method

We verified every representative work named on the map, in the essay and in the deck against arXiv (title, authors, date, abstract), then swept each topic line for 2025-2026 work sorted by submission date. The final list holds 208 entries: 46 core robotics works named by the materials, 94 extended works found in the sweep, and 68 foundation works from the software-side agent, harness and RSI literature, including every paper in the reference lists of the two Lil'Log posts (39 references in the harness post and 21 in the agents post, 54 papers after removing blog and code-repository links) and the Reflexion, STaR, Let's Verify Step by Step, Meta-Rewarding, The AI Scientist, HiSME, BigBang-V1, Gödel Machine and Anthropic "When AI builds itself"items named by the 具身纪元 essay. Non-arXiv and nicknamed items were traced: BigBang-V1 is an Endless Frontier technical report; PRIMO R1 is arXiv 2603.15600 ("From Passive Observer to Active Critic"); VERITAS is arXiv 2606.18247 ("Visual Verification Enables Inference-time Steering and Autonomous Policy Improvement"); HiSME is arXiv 2605.28390 ("You Live More Than Once"); AgenticLab is arXiv 2602.01662 (v1 was titled PLanAR, Purdue); MHS is Anthropic's Model Hardware Standard research preview of 27 August 2026; OpenClawPi is AgileX Robotics'skill library for OpenClaw, not a paper.

Every statement about a paper rests on its abstract or the source text. The report separates "numbers the paper reports"

from "our judgement".

1.3 Scope

Machine translation was constrained by the local machine: the SuperTranslate engine needs an LLM API and no working key was available, so the Chinese versions of the six core papers were produced through the engine's manual-translation path (`export` the text blocks, fill the translations by hand, render in `cache-only` mode, run `inspect` QA). Code as Policies was translated in full for the main body; the other five papers (SkillOpt, CaP-X, ENPIRE, ASPIRE, Harness VLA) have their first page or two (title, abstract, start of introduction) translated. `scripts/translate_papers.sh` completes the rest once an API key is configured.

2. Reading one: "After Harness, what is the next stop for Agent+Robot?"

The essay's value is a testable framework rather than a set of new labels. Its starting point is a counter-intuitive claim: VLA, WAM, Agent and Harness do not replace one another; they are being assembled into one robot system. We check its six "next stops"against the papers.

2.1 Memory: robots have no past

The diagnosis is accurate. A conventional VLA consumes the current image, instruction and proprioception, so the key variable of long-horizon manipulation moves from state to history. The two papers the essay cites check out. RoboMME (ICML 2026) covers 16 tasks across temporal, spatial, object and procedural memory and compares 14 memory-augmented π0.5 variants; its finding is that the usefulness of a memory representation is strongly task-dependent. RoboMME-Interference adds that perceptual memories are best without distractors but decay steadily as unrelated sessions accumulate, subgoal memories gain less but hold up better, and a retrieval step that selects the visually most similar history section restores the no-distractor success rate at every interference level. PonderPounce (August 2026) builds no dedicated memory module: a System 2 MLLM (Ponder) accumulates the episode in its native causal context and asynchronously sends only the newest cognition token and its age to a System 1 VLA (Pounce). It reaches 60.83% on RoboMME with 9B parameters and 50.04% with 0.8B, against 17.93% for current-observation π0.5, with p50 latencies of 78 ms for cognition refresh and 25 ms for action.

The essay predicts four memory types (working, episodic, semantic, skill). Papers from June to August 2026 instantiate each: AGM keeps a subgoal sequence with a progress pointer and advances it only after physical evidence confirms the subgoal, concluding that reliable embodied memory depends on disciplined state updates rather than capacity; Analytic Concept-Centric Memory organises objects by parts, parametric templates, poses and affordances and links transition memory to skill memory; HyMeS ("skills in weights, memory in code") lets a coding agent iterate an executable memory-management system through heuristic learning, lifting RoboMemArena task success from 41.3% to 60.1%; ViReSkill stores verified plans as skills and replays them without further LLM calls.

One point the essay does not develop but the papers do: memory can live inside the VLA. NativeMEM reuses the VLA's own vision encoder to compress each past frame into one token, raising success from 32.4% to 84.0%; LaMem-VLA interleaves short- and long-term memory vaults in the VLA's latent space; Remember Smarter combines Mamba-compressed visual history with a hyperbolic experience space, taking LIBERO-Plus from 53.6% to 70.6%. In-weight and out-of-weight memory are now two parallel routes; Section 7 discusses how they divide the work.

2.2 Reflection: retry is not learning

The essay splits "self-evolution"into seven levels, L1 Retry, L2 Replan, L3 Reflection, L4 Memory Evolution, L5 Skill Evolution, L6 Policy Evolution, L7 Fleet Evolution, and judges that 2026 is moving fast from L2/L3 toward L4-L7. The ladder is the essay's most useful contribution because it places papers directly:

Level	Representative work	What gets updated
L1-L2 Retry / Replan	AgenticLab, Agentic Robot, MALMM, REMAC's pre/post-condition checks	The current plan
L3 Reflection	PhysReflect-VLA, Agentic RAG-VLM's 14-type failure taxonomy	Corrective guidance for the next action
L4 Memory Evolution	Harness VLA learning the frozen VLA's "user manual", AGM, OnEvoMemory	Memory and operating-range rules
L5 Skill Evolution	ASPIRE, ViReSkill, PRACTICE, SkillGLoW, RATs' play-time skill discovery	The skill library
L6 Policy Evolution	LWD, Q-Planning, Z-1, TEMPO, Temporal GRPO	Policy weights
L7 Fleet Evolution	LWD's 16 robots, ENPIRE's 8 stations, RoboOS-NeXT's shared memory	Data and policies shared across the fleet

The essay's summary of Harness VLA deserves emphasis: it neither modifies VLA weights nor expands the skill library; it learns from task-specific execution traces, global success rules and failure models when the VLA is reliable, when to MOVE_TO first, when to re-ground and when to hand off to an analytic primitive. The paper (v4, 2 September 2026) reports 82.4% overall on LIBERO-Pro; the headline +38.6 points is relative to the previous best, RATS (43.8%), and +32.4 relative to the same frozen VLA executed directly (50.0%); RoboCasa365 57.1% vs RLDX-1 30.0% (+27.1; the essay quotes +25.4 from an earlier version); RoboTwin C2R 58.4% vs LingBot-VLA 50.4%. ASPIRE's "compounding experience"also has numbers: skills accumulated on LIBERO-90 raise zero-shot success on LIBERO-Pro Long monotonically with library size, reaching 31% at N=90 against 4% for prior methods.

2.3 Two time scales: the agent owns seconds, the controller owns milliseconds

The essay separates a slow loop (hundreds of milliseconds to seconds) from a fast loop (tens to hundreds of hertz) and cites Fast-in-Slow (117.7 Hz), StreamVLA (72% of timesteps skip autoregressive decoding, 48% lower latency) and LaST0 (latent spatio-temporal chain-of-thought, a low-frequency reasoning expert feeding a high-frequency acting expert). The numbers match the papers.

More important is how it connects the split to harnesses. Harness Engineering for Physical AI argues that a software harness mediates at tool-call boundaries while a Physical AI harness must mediate control, computing and communication simultaneously, because a learned policy's output shifts the trajectory, its inference time shifts the schedule and its payload shifts the bandwidth. It names three missing enforcement functions, Projection (gate each output), Isolation (bound the model's execution and transmission slot) and Transfer (fall back to a verified baseline), and proposes a ROS 2 Harness Profile. The essay's principle "Cloud can think. Edge must survive."has systems evidence from 2026: CloudEdgeVLA keeps 63.8-78.0% success under a 40-step uniform delay window where the compared method reaches at most 6.4%; EcoVLA improves device-edge co-inference energy efficiency by 236% under a 20 Hz constraint; ARLI shows that inference latency breaks the Markov assumption RL relies on and that state augmentation is needed to fine-tune under delay.

2.4 Agent + VLA + RL: it still has to be written back into the weights

The essay's example is USB insertion: after a 3 mm miss the agent can record "correct a little to the left"as a skill, but the continuous relation between visual features, contact state, force feedback and micro-motion belongs in policy weights. The evidence supports this. LWD runs fleet-scale offline-to-online RL on 16 dual-arm robots across 8 real tasks and lifts a single generalist policy to 95% average success, with the largest gains on long-horizon tasks. Q-Planning attaches a small off-policy Q-function to a large BC policy and fine-tunes only the Q-function from deployment failures: stack-cups goes from 40% to 90% and insert-wallet from 25% to 80% on real robots, while SFT on successful rollouts alone stalls at 55% and 30%.

The essay's reading of CaP-X, that the point is not "code beats VLA"but that agentic test-time compute, environment

feedback and RL are starting to combine, matches the paper. Its reading of RoboClaw's Entangled Action Pairs (forward skills paired with inverse recovery skills so the robot resets its own scene, +25% success and 53.7% less human time) also holds, and it names a cost structure that is easy to miss: the most expensive resource in robot RL is people, not GPUs. ENPIRE's Environment module (automatic reset and verification) solves the same problem.

2.5 Real → Sim → Learn → Verify → Real

The essay casts simulation as a safe test environment, the physical agent's sandbox, rather than a return to training everything in simulation. TwinRL fits: it reconstructs a digital twin from smartphone captures, expands the trajectory distribution during SFT, runs parallel RL in the twin to warm up real-world RL and to locate failure-prone configurations, and reaches near-100% success on four tasks with 20 minutes of on-robot interaction. In 2026 the cost of building the twin fell quickly: RoboSnap turns one RGB image into a simulation-ready scene and releases DROID-Sim (564 scenes); Agentic Real2Sim lets a VLM agent convert a recording of real interaction into an episodic twin; PerceptTwin builds an interactive simulation from the robot's perception stack to validate and refine plans, improving GPT-5-family planners by about 39%; SafeDojo runs constrained RL inside an interactive video world model. The world model's role here is, as the essay says, mental rehearsal rather than action prediction.

2.6 Multi-robot: large-scale evolution should not happen on one robot

Of the essay's four sharing layers (shared skill, shared experience, shared world knowledge, shared policy improvement), the papers cover the two ends: RoboOS-NeXT's Spatio-Temporal-Embodiment Memory for shared state and LWD for shared policy improvement. The middle layers have no systematic paper yet; MeCo's similar-task memoisation and FSAR's federated capability registries are early forms. The essay's arithmetic (a million robots × 100 tasks a day = 100 million physical interactions a day) is extrapolation, not data.

A safety note the essay omits: communication attacks on LLM-controlled multi-robot systems turn unsafe information into unsafe actions under all three tested architectures (up to 97.8% unsafe-action success), and a Claim Provenance and Verification Gate cuts the violation rate from 70.0% to 36.6%. Shared experience means a poisoned experience is inherited by the whole fleet.

2.7 Assessment

Four of the six judgements, memory, the reflection ladder, two time scales and simulation as sandbox, are backed by 2026 papers; the Agent+VLA+RL division of labour has two strong pieces of evidence (LWD, Q-Planning); the middle layers of fleet sharing remain a prediction. The essay's most durable outputs are the L1-L7 ladder and the three loops (Execution, Learning, Fleet), which are enough to place most 2026 papers. Its blind spot is safety and governance: Runtime Governance and EmbodiedGovBench do not appear, yet they are prerequisites for a robot that works for a long time.

3. Reading two: the Code-as-Policy → self-evolving robot agent deck

The deck's storyline: Code as Policies (2022) proposes the paradigm → CaP-X (March 2026) turns it into a research platform → ASPIRE (June 2026) adds a skill library → SkillOpt (May 2026) supplies the algorithmic abstraction of self-evolution → ENPIRE (June 2026) puts the coding agent inside a real-robot learning loop. It closes with one abstraction: self-evolution is the optimisation of an external artifact z.

3.1 What each paper evolves

Paper	External state z	Feedback → verification/ selection	Role
Code as Policies (2022.09)	policy code	language instruction + few-shot examples → executability / task completion	proposes the paradigm
SkillOpt (2026.05)	skill.md	scored rollouts → accept only if the held-out validation score strictly improves	algorithmic abstraction
CaP-X (2026.03)	CaP agent / benchmark	execution feedback / visual differencing → CaP-Bench / sim2real	research platform
ASPIRE (2026.06)	control programs + skill library	multimodal traces → validated fixes / task success	experience reuse
ENPIRE (2026.06)	training code / recipes / infrastructure	real rollouts + logs → physical task success	real-robot algorithm engineering

Key numbers: Code as Policies lifts HumanEval P@1 to 39.8% with hierarchical code generation and keeps 62-80% success on simulated tabletop tasks with unseen attributes and instructions while CLIPort drops to near zero. SkillOpt is best or tied on all 52 (model, benchmark, harness) cells across 6 benchmarks, 7 models and 3 harnesses, adding +23.5 points in direct chat, +24.8 inside Codex and +19.1 inside Claude Code on GPT-5.5. CaP-X evaluates 12 models, shows success falling as human abstractions are removed, and recovers near-human reliability training-free with CaP-Agent0; CaP-RL adds RL with verifiable rewards and transfers sim-to-real with a small gap. ASPIRE gains up to 77% on LIBERO-Pro, 72% on Robosuite bimanual handover and 32% on BEHAVIOR-1K, and reaches 31% zero-shot on LIBERO-Pro Long against 4%. ENPIRE lets frontier coding agents train policies to 99% success autonomously and cuts pin-insertion convergence from over 1.5 hours to about 40 minutes on an 8-station fleet.

3.2 Heuristic Learning and the unifying formula

Slide 2 defines Heuristic Learning (HL): a coding agent iteratively edits software structure. HL shares the state-action-feedback-update loop of deep RL but updates software instead of network parameters; feedback is digested by the coding agent and can come from environment reward, tests, logs, video, replay or humans; updates bypass backpropagation because the agent edits the policy, state detectors, tests, configuration or memory directly; the maintained object is a Heuristic System (HS), which contains at least a programmatic policy, a state representation, feedback entry points, experiment records, replay or tests, memory and the update mechanism.

Slide 24 compresses this into $z_{t+1} = A(z_t, \tau_t, r_t, \log_t)$: A is the LLM/VLM coding agent, z can be a skill document, `policy.py`, a skill library, a training recipe or a git branch, τ is the rollout trajectory, r the verification signal and log the diagnosable evidence (errors, visual diffs, robot traces, training curves). The formula puts five different-looking papers into one coordinate system: they differ only in what z is, where r comes from and what A is allowed to edit.

SkillOpt pushes the analogy to its strictest form: parameter ↔ skill document, gradient direction ↔ edit direction extracted from trajectories, learning rate ↔ edit budget L_t, validation ↔ held-out selection gate. Each design choice has an ablation: any moderate edit budget (L_t = 2-8) beats unbounded rewriting, and removing it drops Spreadsheet 77.5 → 75.7 and LiveMath 61.3 → 57.3; the gate keeps only 1-4 of dozens of proposed edits per epoch, and OfficeQA's +39 points came from a single accepted edit; removing the rejected-edit buffer drops Spreadsheet 77.5 → 72.9; removing epoch-wise slow/meta updates drops Spreadsheet 77.5 → 55.0, the largest degradation in any ablation. SkillOpt-Lite later reduces the pipeline to a minimum from a zeroth-order-optimisation viewpoint and generalises it to whole harnesses (HarnessOpt), letting GPT-5.4-nano beat GPT-5.5 on SpreadsheetBench.

3.3 Four lessons from ENPIRE

1. **Simulation success is not real-world robustness.** In Gym-PushT, Codex and Claude Code reach 95% in about 2 hours and Kimi takes twice as long; on the real Push-T, Codex approaches 95% while Claude and Kimi stall near 75% and 60%, because contact friction drifts, object poses carry error, the controller differs from real dynamics and heuristics are sensitive to corner cases. The follow-up experiments therefore let the agent mix heuristics, BC and RL. Goldberg's group offers the mirror image: Claude Code, with no demonstrations, writes an algorithmic Push-T solver that reaches 100% in simulation with 46% fewer steps than the best diffusion policy trained on 200 human demonstrations. In simulation the coding agent has already won; on hardware it has not.
2. **Pure BC is not enough.** Pin insertion requires 50 consecutive real successes. BC learns only the demonstration distribution; iterative BC lets the current policy collect drifted states, failed contacts, recovery motions and new successes on the robot and retrains to cover the policy-induced distribution. The agent tried BC, iterative BC, online rollout aggregation, offline RL, online RL, offline-to-online RL and RL with BC regularisation, plus batch size, update frequency and BC-weight tuning; the largest step on the hill-climb was BC regularisation (+10.8 points).
3. **Fleets raise hypothesis throughput.** With 1/4/8 agents on 1/4/8 robots, time to a normalised Push-T score of 1.0 falls from about 5 hours to about 2, and pin insertion from over 1.5 hours to about 40 minutes. Eight times the robots gives two to three times the speed because agents re-explore similar ideas, wait for training or model outputs, read other branches, merge experiments and analyse logs. What a fleet multiplies is hypotheses, not rollouts of one policy.
4. **Continual learning happens in text.** After several agents accumulated experience on pin insertion, the authors had them write a Markdown research summary (which RL methods are stable, whether BC regularisation matters, which batch sizes work, what compensation contact tasks need from the controller, which failure modes to check first), seeded the GPU-insertion agent with it, and deliberately deleted raw trajectories, checkpoints, hidden logs and the old workspace. The transfer path is research experience → textual memory → new autoresearch, not continual learning of weights. This is the same mechanism Lilian Weng calls "file system as persistent memory" .

A fifth experiment (slide 21) settles the division of labour between code policies and VLAs: in RoboCasa365 the pure GR00T VLA often lacked a stable pre-grasp positioning motion, so the agent called detection, segmentation, 3D localisation and motion planning to move the end-effector to a hover pose and then handed control to the VLA for the contact-rich grasp. Averages: GR00T N1.5 about 53%, ENPIRE's hybrid about 77%, CaP-X about 27%. Harness VLA and BATON adopt the same split: code for geometrically determined, plannable phases; the VLA for contact-rich phases.

3.4 What the deck does not yet cover

The deck's timeline ends in late June 2026. July to September produced three continuations. First, more abstract reuse units than ASPIRE's skill library: PRACTICE trains a skill learner that issues add/refine/merge/remove batch edits to a persistent library while the executor stays frozen; SkillGLoW argues that the unit of reuse is the solving procedure shared by a family of related tasks, and its unmodified library lifts unseen ALFWorld success from 73.9% to 83.9%. Second, optimisation of the runtime itself: Zetta evolves code-based runtime critics and recovery skills online with the base policy frozen, using three timescale-separated loops for action-frequency governance, rollout-level critic-recovery proposals and validation-gated skill updates, reaching 90.8% on LIBERO-Pro and 93.6% on RoboCasa with an 11.1× inference speedup; SHAPER lets one frozen model act as planner and optimiser of skills and a context-code harness. Third, RHO: coding agents search multi-file policy repositories (Repositories-as-Policies) at training time and execute single-turn at deployment, 45.0% on LIBERO-PRO against 12.83% for $\pi 0.5$ and a new best of 70.0% on Robosuite. RHO answers the long-standing objection that multi-turn code generation is unfit for real-time control.

Slide 9's caution still stands: CaP optimises the program-interface layer, not a continuous controller. ENPIRE's hybrid and RHO's "same low-level primitives"both show that in 2026 Code-as-Policy no longer competes with VLAs; it writes their surroundings.

4. Reading three: the 23 topic lines of the retrieval map (status as of September 2026)

For each line: the problem, what the map's representative works say, what 2025-2026 added, and our judgement. Full titles, links and authors are in the corresponding `README.md` section.

T1 Agent + Robot overview. Problem: connect LLM/VLM reasoning to a real arm in a perceive → decompose → execute → verify → replan loop. AgenticLab (arXiv 2602.01662) defines the VLM's reasoning space with a planning language (object predicates, action schemas with preconditions and effects, symbolic plans as executable intermediates) and checks symbolic effects against onboard observations after every action. Agentic Robot coordinates a reasoning model, a VLA executor and a temporal verifier through a Standardized Action Procedure (79.6% on LIBERO long-horizon). ManiAgent reaches 86.8% on SimplerEnv with multi-agent collaboration and uses the agent to generate VLA training data. 2026 added Cortex (32 canonical skill primitives with a bidirectionally aligned planning interface), HoloAgent-0 (Embodied AgentOS plus 3D spatial memory), VoLo (a VLM that steers a VLA/WAM as an interruptible tool mid-rollout, naming the timing problem "physical orchestration") and Guava (a systematic exploration of the harness design space yielding three ingredients: iterative perception-reasoning-action loops, semantic action abstractions, multimodal observations, distilled into a 4B model). "Agentic AI for Robot Control: Flexible but still Fragile"is the sober counterpoint: transfer between two real platforms required only a new system prompt, yet non-deterministic behaviour, instruction-following errors and prompt sensitivity were pervasive. Judgement: the loop skeleton is standard; reliability is limited by verification and grounding, not planning.

T2 Coding agents controlling robots. Three branches grew out of Code as Policies. Test-time generation: CaP-X's CaP-Agent0, MALMM's Planner/Coder/Supervisor agents, and Neuro-Symbolic Code-as-Policies (NeurIPS 2025 Spotlight, +46.2% over CaP with symbolic and interactive validation). Training-time search: RHO's policy repositories, MEMENTO's memory-guided memetic evolution with an evolved rollout evaluator, PhysCaP's physics-informed exploration that estimates mass and stiffness from proprioception. Research automation: ENPIRE, HARBOR (robot RL as a harness-engineering problem), Nautilus (one prompt to reproduction, evaluation, fine-tuning and deployment workflows), AgenticRobotics (a control plane with an evidence-graded skill library and a signed verifier). Judgement: the coding agent's role has moved from "the one who writes the policy"to "the one who runs the experiments"; Lil'Log's autoresearch has entered the physical world.

T3 OpenClaw / ROS. Two papers share the name ROSClaw. Cardenas et al. connect the OpenClaw agent runtime to ROS 2 with capability discovery and affordance injection, observation normalisation, pre-execution validation inside a configurable safety envelope and audit logging; swapping models or platforms is a configuration change, and the substrate doubles as an instrument: frontier models differ up to 4.8× in out-of-policy action proposal rates, and executive-layer design affects task completion and safety more than prompt wording. Zhao et al.'s ROSClaw targets heterogeneous robots with e-URDF constraints and a unified VLM controller spanning collection, training and execution. AgentRob bridges forum agents and Unitree Go2/G1 through MCP (and exposes hijacking risks); OpenGo is an OpenClaw-driven Go2 with a skill library and feedback-based self-learning; OpenClawPi is AgileX's skill library. MCP is becoming the robot-side universal interface: ROSBag MCP Server, ChemBot's MCP sub-agent orchestration, Contract-Grounded BT Synthesis (the coding agent fetches a skill contract from a robot-side MCP server before synthesising a behaviour tree). Judgement: the model-agnostic executive layer is the most practical infrastructure advance of 2026; it turns safety envelopes and audit into configuration.

T4 Long-horizon tasks. RoboClaw's Entangled Action Pairs enable self-resetting data collection (+25% success, 53.7% less human time); H-WM couples a logical and a visual world model to guide VLAs; REMAC's pre/post-condition checks and scene-specific self-evolution raise multi-robot success by 40%. The most important 2026 addition is BATON, which shows the "frozen VLA + LLM agent"recipe breaks twice on long horizons: whole-task exploration costs T^K across K stages and a failure cannot be attributed to a stage, and VLA primitives carry an exit condition but no entry condition. BATON explores per subtask (cost $T \cdot K$) with a transition-aware memory governing invocation, handoff and lookahead transitions (+11.6% task success on RoboMemArena). AtomBridge inserts LLM-generated transitional actions at skill boundaries (+10-25% on 8-step sequences); Foresight Residual RL shapes handoff-state quality with an offline foresight value (85.6% vs 54.5% on a three-phase assembly). Judgement: the long-horizon problem is being restated as a skill-handoff problem; entry conditions, exit conditions and handoff quality are the core

variables.

T5 Robot memory. See Section 2.1. Within the line: in-weight memory (NativeMEM, LaMem-VLA, Remember Smarter, Dual Latent Memory, VQ-Memory, SkillMemo) versus out-of-weight memory (AGM, HyMeS, BATON, Analytic Concept-Centric Memory, OnEvoMemory, ChemBot's dual-layer memory, RoboHarness's execution memory). RoboMME's finding that representation effectiveness is task-dependent is the most reliable empirical fact on this line; AGM's conclusion that reliability comes from update discipline rather than capacity is the most useful design guidance. Memory Anchors adds a continual-learning angle: 10% of replay experiences anchor past performance, and excluding them raises catastrophic forgetting 4.5×.

T6 Reflection and failure correction. REMAC, AgenticLab and ASPIRE sit at L2, L2-L3 and L5. 2026 gave reflection finer forms: PhysReflect-VLA's feasibility operator, action-explanation operator and LLM reflection module (+5.4% on contact-rich real tasks); Agentic RAG-VLM's 14-type failure taxonomy and three-level retry (cluttered grasping from 25% to 78.3%); PhysiAgent organising monitor, memory and reflection components from the VLA's real-time proficiency feedback; Online Continual RL with World Model Feedback using DreamerV3 prediction residuals to detect OOD events and trigger fine-tuning. Judgement: reflection is worth only what it writes into a persistent object (memory, skill, weights); otherwise it is a more expensive retry.

T7 Self-evolution. Arcadia argues embodied learning is a lifecycle problem whose four stages are non-decomposable. PhyAgentOS delivers scheduling, verification, memory, benchmarking and safety as system services, schedules sessions rather than actions, decouples cognition from execution through State-as-a-File, separates execution termination from semantic completion with a SessionVerifier, and consolidates verified outcomes into epistemic memory, validated on 19+ embodiments. Growing with Your Embodied Agent takes the human-in-the-loop route, encoding corrections as reusable skills with external memory and RAG to solve a "build a house"task over 20+ primitives. August 2026 refined the object of evolution: PRACTICE (a trained skill learner maintaining the library), SkillGLoW (procedural families as the unit of reuse), Zetta (runtime critics and recovery skills), SHAPER (skills and a context-code harness), Motus2 (one model exposing policy, simulator and evaluator interfaces), Q-Planning (fine-tune only the Q-function). Against the software-side Self-Harness, AHE and Meta-Harness, robot-side self-evolution is more conservative about the editable surface (almost everything freezes the base policy) and more dependent on physical evidence for verification. Judgement: the real threshold is the validation gate and the editable surface, not the search algorithm.

T8 Skill library. Agentic Skill Discovery (2024) is fully LLM-driven: the LLM proposes tasks, samples reward and success functions, RL learns policies and a VLM verifies, growing a library from zero. Atomic Skill Library builds a dynamically expandable atomic library through VLP decomposition → skill abstraction → VLA fine-tuning. ViReSkill replays verified plans without LLM calls. Three 2026 changes matter: ASPIRE's library persists across tasks, sim/real and embodiments with a monotonic size-versus-zero-shot curve; RATs adds a "play"stage that discovers skills before downstream tasks arrive (+20.6 points over CaP-Agent0 on LIBERO-PRO, and the skills plug into other Code-as-Policy agents by retrieval); VASO represents skills as semantic contracts with a formal interface, turning model-checking counterexamples into textual gradients (97.2% specification compliance) and addressing, for the first time, the cost of trusting a skill. ARCHITECT distils human language corrections into a persistent library; SkillMemo segments skills implicitly with MoE gating. Judgement: skill libraries are turning from collections of code snippets into knowledge bases with verification evidence and operating ranges.

T9 VLA + RL. TwinRL, LWD, SAC Flow, CaP-RL and TT-VLA span training-time to test-time RL. 2026 concentrated on two technical points. Credit assignment: Temporal GRPO assigns stage-wise advantages to fix trajectory-level credit aliasing; TEMPO freezes the VLM backbone and updates the semantic projection slowly and the action expert quickly; WCM trains the critic to predict future latents as well as value (state of the art on 149 tasks). Reward-free operation: T²VLA uses generation confidence as intrinsic reward; RL²-VLA activates latent compositional steering only when failure is predicted (+17.3% OOD); Z-1 pushes π0.5 to 80.6% on 24 RoboCasa tasks with task-wise GRPO. ARLI addresses deployment: inference latency breaks the Markov assumption and standard RL fails outright under delay. Judgement: VLA+RL moved from "can it work"to "how to assign credit and how to run without rewards and under latency"; this is a mature engineering line.

T10 Deployment data flywheel. LWD's Deploy → Experience → RL → Updated Policy → Redeploy loop is the most

complete evidence; RoboClaw solves self-reset; Arcadia closes the loop through sim-from-real evaluation. 2026 added RoboGene (diversity-driven, self-reflective task generation, 18k trajectories, better VLA pretraining), AgenticRobotics (operator absence as an operational claim backed by evidence-gated promotion, recoverable state and a hardened false-promotion rate of 0.001) and Q-Planning's real-robot self-improvement curve with BC frozen and no human intervention. Judgement: the bottleneck moved from "cannot collect" to "which experiences deserve to flow back", which is again a verifier question.

T11 Digital twin / sim2real. The cost of building a twin is falling fast: RoboSnap (one RGB image, sim-real correlation, DROID-Sim), Agentic Real2Sim (open-weight VLM agents converting recordings into episodic twins), ConCent (contact-event sequences as the learning objective to stop policies exploiting unrealistic simulator contacts), BestMan (automated scene generation plus hardware-agnostic middleware), Real2Sim via Active Perception (VLM-generated behaviour trees acquiring missing physical parameters), PerceptTwin (plan validation from the perception stack). Judgement: the twin has become part of the verifier; its value is running an agent's new skill in physics before hardware.

T12 World model. H-WM guides VLAs with a logical-plus-visual hierarchy. 2026 clarified the role: Motus2 exposes policy, simulator and evaluator from shared weights; WCM adds a world-modelling objective to the critic; ContactGuard predicts and aborts likely failures before contact from a latent world model; SafeDojo trains safe actions in imagination; Online Continual RL uses prediction residuals for OOD detection. Judgement: the most useful role of a world model in an agentic robot is rehearsal and monitoring, not action generation, consistent with the essay's "mental rehearsal".

T13 Fast-slow dual systems. Fast-in-Slow (System 1 inside System 2 with shared parameters, 117.7 Hz), OneTwoVLA (one model switching reasoning and acting modes), StreamVLA (slow thinking only at sub-task transitions, imagining a completion state as a time-invariant goal anchor, 98.5% on LIBERO), LaST0 (latent spatio-temporal CoT with heterogeneous-frequency experts, +13-14% on 10 real tasks), RationalVLA (rejecting defective instructions on the six-dimension RAMA benchmark). 2026 added UniFS (VLM layers stratified by update frequency, 36.5 ms → 17.8 ms), Latent Bridge (predicting inter-step VLM feature deltas, 50-75% fewer VLM calls), Libra-VLA (asynchronous coarse-to-fine), VisualThink-VLA (visual intermediate reasoning, 8.377 s → 0.367 s per step) and DSWAM (a world-action model as System 1). Judgement: the design space is mapped (OpenHelix surveys it); the open question is when to switch, with StreamVLA's completion-state gate and RL²-VLA's failure-prediction gate as two answers.

T14 Edge agent / on-device deployment. Harness Engineering for Physical AI supplies the framework (Projection / Isolation / Transfer). Systems work in 2026: PhyAI unifies VLA/WAM inference across onboard, edge and cloud with 1.40-4.65× speedups and a control-time Roofline separating inference-bound from environment-bound control; EcoVLA optimises device-edge energy; CloudEdgeVLA treats temporal misalignment as representation learning; ST-Merge merges spatio-temporal visual tokens training-free (8.3× on $\pi 0.5$ at 1024^2); Habilis-β proposes the Productivity-Reliability Plane (Tasks per Hour × Mean Time Between Intervention) under one-hour continuous runs. Judgement: evaluation is shifting from single-trial success to continuous-run throughput and intervention intervals, the metrics that matter for a robot that works all day.

T15 Harness. The densest line of June-August 2026: RHO (policy repositories), Harness VLA (operating ranges of frozen primitives), Harness Engineering for Physical AI (middleware as harness), PhyAgentOS (harness as OS), Thea (Scene Graph as Context, Evaluation as Exit Codes), Guava (design-space study), HARBOR (RL engineering), Zetta (self-evolving closed-loop harness), SHAPER (skill-harness evolution), RoboHarness (capability boundaries from execution memory, Memory Bridge to in-distribution regions), Agentic Harnesses (an LLM-as-a-Judge verification layer between planning and execution, 97% adversarial containment), Cortex (inference-time harness engineering). Software-side methodology comes from Recursive Harness Self-Improvement, Meta-Harness, Self-Harness and AHE. Judgement: harness is the centre word of Agent × Robot in 2026, with the standing caveat that "harness updating is not harness benefit".

T16 Runtime. PhyAgentOS provides session-level scheduling, preflight, supervised execution, evidence collection and acceptance. Runtime Governance externalises governance into a dedicated layer (policy checking, capability admission, monitoring, rollback, human override): 96.2% interception of unauthorised actions over 1000 trials, unsafe continuation under drift cut from 100% to 22.2%, 90.7% recovery success; against AutoRT-style constitution

filtering and RoboGuard-style two-stage guardrails, pre-execution filtering is comparable across methods, but only this framework offers continuous runtime detection (RVDR 61.3% vs 0%) and structured recovery. The same group's EmbodiedGovBench defines seven governance dimensions; ICAN-Deploy verifies identity-stable canary deployment with TLA+; FSAR argues fleet coordination should federate coherent single-agent runtimes rather than fragment each robot into agent societies. HoloAgent-0's Embodied AgentOS, PhyAI's runtime and AgenticRobotics'control plane belong here too. Judgement: the runtime is where "governable"becomes engineering; seven-dimension governance metrics track deployers'concerns better than task success.

T17 Safety. Beyond ROSClaw's envelopes, Runtime Governance's interception and RationalVLA's instruction rejection, 2026 added four kinds of work. Guardrail models: EMBGuard (ICML 2026), 2B/4B models judging (observation, action) pairs for physical risk with fewer false positives than proprietary MLLMs. Formal methods: SENTINEL's temporal-logic evaluation at semantic, plan and trajectory levels; VASO's verification-guided skill evolution. World models: SafeDojo, ContactGuard. Deployment view: "Same Weights, Different Robot"shows that action-unnormalisation metadata is part of the executable policy; substituting one plausible metadata key drops LIBERO-Goal from 28/28 to 2/28, so safety review of a checkpoint misses the policy that reaches the controller. Multi-robot communication attacks and AgentRob's hijacking risk complete the picture. Judgement: safety is expanding from filtering unsafe plans to continuous runtime detection, recovery, auditability and upgrade safety, and has begun to reach previously unexamined layers such as action-space semantics.

T18 Standard interfaces / hardware API. Anthropic's Model Hardware Standard (research preview, 27 August 2026) is the landmark: a shared specification letting agents discover, operate and troubleshoot arbitrary devices (microscopes, liquid handlers, cameras, robotic arms, laser systems), described in coverage as "MCP for hardware"and callable through MCP, CLI or code. ROSClaw's capability discovery, Contract-Grounded BT Synthesis's robot-side MCP contracts, ROSBag MCP Server, AgentRob's MCP bridge and the ROS 2 Harness Profile are concrete instances. Judgement: interface standardisation decides how portable harness layers are; whether MHS and the ROS 2 Harness Profile converge is worth tracking.

T19 Multi-robot collaboration. RoCo (2023) negotiates roles and waypoints in natural language and hands them to a multi-arm motion planner; REMAC adds condition checks and self-evolution; RoboOS coordinates embodiments through a Brain-Cerebellum architecture and real-time shared memory; RoboOS-NeXT adds STEM memory for lifelong collaboration. 2026: DynaHMRC (decentralised role-aware agents with leader election), Scale-Plan (LLM-guided action-graph search that prunes PDDL problems), MeCo (similar-task memoisation), Tool-RoCo (agents rarely invoke other agents as assistants: cooperative tools were 7.09% of calls), World-Model Alignment Through Dialogue (dialogue cuts action conflicts by 40-83 points yet lowers task success), FSAR (federation over fragmentation). Judgement: the language layer of multi-robot collaboration is not short of methods; what is missing is consistency of shared memory and trustworthiness of communication.

T20 Cross-embodiment. RoboOS supports heterogeneous embodiments; ASPIRE's skills transfer across embodiments and robot APIs with initial sim-to-real evidence; Harness VLA reaches 58.4% on RoboTwin C2R; ROSClaw (heterogeneous) uses e-URDF constraints; BestMan uses hardware-agnostic middleware; PhyAgentOS validates on 19+ embodiments. Judgement: cross-embodiment is easier at the agent layer than at the policy layer, because code, skill contracts and harness configuration transfer naturally while policy weights do not.

T21 Fleet learning. LWD is the policy-level evidence, RoboOS/RoboOS-NeXT the memory-level evidence, ENPIRE's 8-station fleet the research-level evidence (with Mean Robot Utilization and Mean Token Utilization as efficiency metrics); FSAR discusses fleet-level governance and recovery boundaries. Judgement: fleet learning has both ends (shared policy, shared state memory) but lacks systematic work on shared experience and shared world knowledge in between; speedups are sublinear because the bottleneck is hypothesis generation and analysis, not rollouts.

T22 Active perception. AgenticLab's online verification requires active observation; PhysCaP lets a Code-as-Policy agent infer mass and stiffness through interaction (finding hidden objects, detecting empty cans, picking ripe avocados) with a Planner deciding when to explore and stop and a Prioritizer filtering implausible interactions; ActiveVLA localises critical regions and selects viewpoints with 3D zoom; Real2Sim via Active Perception acquires missing physical parameters with VLM-generated behaviour trees. Judgement: in agent systems active perception takes the form of "exploration as a callable tool"; cost control (when to stop) is the key.

T23 Verifier / success verification. Harness VLA's success rules and failure models, PhyAgentOS's SessionVerifier and REMAC's condition checks are the map's three forms. New 2026 work shows the line has become the bottleneck: Thea's Evaluation as Exit Codes; AGM's 2.43M-parameter verification head deciding when to verify from proprioceptive cues and what was achieved from point tracking and cross-view language comparison; Agentic Harnesses'LLM-as-a-Judge ensemble; LLM-as-a-Verifier treating verification as a scaling axis, computing expected scores over token logits (87.4% on RoboRewardBench) and serving as dense RL reward; VASO replacing trace-level evidence with model checking; PerceptTwin validating plans in simulation; Consilience on verifier-free selection from confidence trajectories. The position paper "VLA Cannot Be Verified to Perform Physical Reasoning"argues that success rate cannot separate semantic matching from physical generalisation and calls for controlled-variation evaluation. Judgement: the verifier sets the ceiling of every self-evolution loop; reward hacking, over-optimism and "100% in sim, 60% on hardware"all happen here.

T24 Robot RSI: recursive self-improvement (new line). Problem: can a robot system take part in improving "how it gets better"and carry the result into the next round. The line comes from the 具身纪元 essay (Section 6) and sets the software-side RSI lineage (Gödel Machine's "modify only when provably beneficial", STaR, Reflexion, Let's Verify Step by Step, Meta-Rewarding, The AI Scientist, HiSME, BigBang-V1, Anthropic's "When AI builds itself") beside four robot-side stages: deployment-time self-evolution (ASPIRE, RoboHarness, Zetta), training-time self-iteration (RoboClaw, LWD, Q-Planning), self-evaluation (PRIMO R1's process-level video critic, VERITAS's inference-time visual verifier) and auto research (the Eureka → DrEureka → ENPIRE progression from rewards to simulation parameters to the whole research loop). Judgement: Robot RSI is the frame that strings T2, T7, T9, T10 and T23 together rather than a new method; its two distinctive problems are a repeatable, scalable physical verification environment and keeping the evaluator outside the loop when the evaluator itself is evolving.

5. Reading four: Lilian Weng's two posts

5.1 "LLM Powered Autonomous Agents"(June 2023)

The decomposition, an LLM brain plus Planning (subgoal decomposition, reflection and refinement), Memory (short-term as context, long-term as an external vector store with fast retrieval) and Tool use (external APIs), is the direct ancestor of the essay's full architecture diagram. The essay places Planning in the Agent/System 2 layer, splits Memory into Memory/Skill/Dataset, and concretises Tool use into the Harness/Runtime's tool routing and the Skill Layer's four primitive types (code skill, analytic primitive, VLA primitive, RL policy). The 2023 post's three challenges, finite context length, difficulty of long-term planning and task decomposition, and unreliability of the natural-language interface, map onto the robotics lines for memory, long-horizon tasks and verification.

5.2 "Harness Engineering for Self-Improvement"(July 2026)

The post defines a harness as "the system surrounding a base model that orchestrates execution and decides how the model thinks and plans, calls tools and acts, perceives and manages context, stores artifacts, and evaluates results". Its three design patterns each have robot counterparts.

Lil'Log harness pattern	Robot-side counterpart
Workflow automation: a goal-oriented plan → execute → observe/test → improve loop	ENPIRE's reset → execute → verify → refine; ASPIRE's execution engine → diagnose → repair → validate; RoboClaw's single agent loop over Collection ↔ Learning ↔ Deployment
File system as persistent memory: durable state and artifacts in files rather than in context	PhyAgentOS's State-as-a-File (cross-layer state as Markdown + YAML); ENPIRE's Markdown research summaries; AgenticRobotics' commit-keyed crash recovery and evidence-graded skill library
Sub-agents and backend jobs: parallel hypothesis search, job monitoring, result merging	ENPIRE's agent team × robot fleet; HARBOR's staged specialist agents; RATs' Robotics Agent Teams

The post's optimisation ladder, instruction prompts → structured context → workflow → harness code → optimizer

code, aligns exactly with the deck's z: SkillOpt at the structured-context level (skill.md), ASPIRE at workflow and library, RHO and SHAPER at harness code, Meta-Harness and SkillOpt-Lite's HarnessOpt at optimizer code.

Two of its findings matter especially for robots. First, STOP's self-taught optimiser improved with GPT-4 but degraded with GPT-3.5 and Mixtral, and Lin et al. (2026) separate two axes: the ability to produce useful harness edits (roughly flat from Qwen3.5-9B to Claude Opus 4.6) and the ability to benefit from an updated harness (non-monotonic, mid-tier models gain most). The Codex/Claude/Kimi gap on ENPIRE's real Push-T is more likely a harness-benefit gap than a harness-updating gap. Second, the evaluator and permission control must sit outside the evolution loop: AHE makes the runs directory, tracer, verifier and LLM configuration read-only so that every recorded gain is attributable to a harness edit rather than reward hacking. Runtime Governance, ICAN-Deploy and AgenticRobotics'signed verifier do the same on the robot side.

Every paper cited by the two posts is collected under the Foundations line T0 of the README (39 references in the harness post, from Good 1965 and Yudkowsky 2008 on the RSI concept, through the self-play line of Absolute Zero, Self-Rewarding and SPIN, the search methods ADAS, AFlow, ShinkaEvolve and ThetaEvolve, to the AI-R&D benchmarks PaperBench, RE-Bench, MLE-bench and KernelBench; 21 in the agents post, including CoT, ToT, ReAct, Reflexion, Toolformer, HuggingGPT and Generative Agents), with the RSI-related ones also listed under T24.

The post's seven RSI challenges, weak and fuzzy evaluators, context and memory lifecycle, negative results, diversity collapse, reward hacking, long-term success, and the role of humans, take concrete robotic forms: weak evaluators become the Verifier line; memory lifecycle becomes RoboMME-Interference's decay under distraction; negative results become the failure trajectories ASPIRE and Zetta treat as their most valuable data; diversity collapse becomes ENPIRE's agents re-exploring similar hypotheses; reward hacking becomes the Push-T that is 100% in simulation and 60% on hardware; the role of humans becomes the interventions in LWD and TwinRL's human-in-the-loop rollouts.

5.3 What the posts do not cover

Lil'Log's harnesses live in the digital world: state is readable, outcomes are judgeable, failures can be retried a thousand times. Thea states the difference most clearly: the physical world withholds two abilities software grants for free, reading the state of the world and judging the outcome of an action, and it fills them with Scene Graph as Context and Evaluation as Exit Codes. Harness Engineering for Physical AI fills a third gap: time. A software harness does not care that an inference took two seconds; a robot harness must, because those two seconds change the control schedule and the trajectory. These three points are the essential difference between robot harnesses and software harnesses, and the starting point for several insights in Section 7.

6. Reading five: "GPT-6 may not be a good robot brain, but it could help Wang Xingxing accelerate Robot RSI"(具身纪元)

The essay (Marilyn Liu, WeChat account 具身纪元, August-September 2026) starts from a contrast: in Robocurve's third-party test GPT-6 Astra scores 19/20 on "put the block in the bowl"(Claude Fable 5.1: 8/20), yet both score 2/20 on fine puzzle insertion, and turning a washing-machine knob took GPT-6 Astra four minutes. The author's reading: the model upgrade improved visual understanding, spatial planning, error correction and tool use, but where contact, friction, precise alignment and millimetre errors are involved, the LLM's gains did not turn into control ability. Hence the proposal to watch not the LLM as policy generator but the LLM as an engine of recursive self-improvement (RSI) for embodied systems. This is the same observation as our I2 (frozen VLA plus external learning has a ceiling; contact-related continuous relations must go into weights), seen from the other side.

6.1 The definition of RSI and its two axes

The essay unpacks RSI into three words: Self (what changes can be the answering style, memory, tools, code, training data, evaluator or weights, not necessarily the whole source code), Improvement (the new version must be better on a checkable objective; an unverifiable change is only a change) and Recursive (last round's product enters the next round

and helps the system build the following version more effectively: "the ability to improve is itself used to improve"). It then proposes two axes, the most useful tool in the piece:

Axis 1: what is improved	LLM-side examples	Robot-side examples named by the essay
Deployment-time self-evolution: weights fixed; reasoning, memory, tools, code and harness change during tasks	Reflexion (reflections stored in memory and reused, HumanEval 91%); HiSME (meta-skills for "how to generate and edit skills" learned from execution traces, MineDojo 0.700 → 0.856); Lil'Log's harness-first thesis	ASPIRE (validated fixes stored as skills); RoboHarness (execution memory orchestrates heterogeneous policies and moves the robot into the next policy's familiar state first; 135 real-robot trials)
Training-time self-iteration: last round's data, reasoning, preferences or rewards go back into training	STaR (keep correct rationales, re-derive wrong ones from the answer, fine-tune the next model); BigBang-V1 (proposer / critic / meta-critic synthesise about ten thousand verifiable problems to update weights)	RoboClaw (learn both "complete the task" and "restore the scene", alternate them and harvest successful data; 53.7% less human time)
Self-evaluation: improve the judge, reward model, process reward model or verifier	Let's Verify Step by Step (process reward model locating the faulty step); Meta-Rewarding LMs (one model as actor, judge and meta-judge; AlpacaEval 2 LC 22.9% → 39.4%)	PRIMO R1 (video anchored between initial and current frames to judge progress and where failure began; 67% on RoboFail); VERITAS (frozen generalist policy plus gradient-free visual verifier; 50 verified autonomous trajectories give 70% vs 65% for the same number of human demonstrations)
Auto research: propose hypotheses, change algorithms, run experiments, analyse results, schedule the next round	The AI Scientist (from code template to automated review)	ENPIRE (automatic reset, success verification, policy update and real trials packaged as tools); Eureka (LLM-written rewards beat human rewards on 83% of tasks); DrEureka (LLM writes rewards and domain-randomisation ranges; quadruped balances on a yoga ball and transfers to hardware)

Axis 2 is the degree of human involvement: human-in-the-loop (the AI proposes, a human confirms each step), human-on-the-loop (data, rewards, evaluation and execution run largely automatically; humans supervise results, set permissions and control releases) and closed loop (the system proposes, verifies and adopts improvements within a pre-authorised scope). The essay does not place papers on this axis, but the Runtime Governance, ICAN-Deploy and AgenticRobotics work in our T16 is exactly the infrastructure needed to move from on-the-loop to closed loop.

6.2 Mapping onto this report's framework

The two axes stack directly onto the earlier chapters. The four stages of axis 1 correspond to what the coding agent A in the deck's formula is allowed to edit: deployment-time self-evolution edits the harness, memory and skills inside z; training-time self-iteration edits policy weights; self-evaluation edits the source of r; auto research edits the whole experimental procedure. They also map onto the essay's L1-L7: deployment-time self-evolution covers L3-L5, training-time self-iteration is L6, and auto research turns L7's fleet experience into the research process itself. The essay's summary of ENPIRE, "reset, success verification, policy update and real-robot trials packaged as tools so that a coding agent can run execute-check-edit-execute continuously", matches Section 3.3.

The essay adds three things. First, by placing PRIMO R1 and VERITAS under self-evaluation it supplies two verifier types missing from our T23: a process-level video critic (judging how far the task has progressed and where it started to fail, rather than whether the final frame looks like success) and an inference-time action verifier (a generator-verifier framework whose verified rollouts become fine-tuning data with efficiency comparable to expert demonstrations).

Second, it reconnects the 2023-2024 Eureka / DrEureka line ("LLM writes rewards and simulation parameters") to auto research, giving ENPIRE-style physical autoresearch an earlier ancestry: automate the training method first (rewards, domain randomisation), then the whole research loop. Third, it states the core difference between robot RSI and LLM RSI: evaluation has two parts, whether the evaluator can judge success, progress and failure location, and whether the system can provide a repeatable, scalable environment in which old and new policies are verified under comparable conditions; LLMs never faced the second problem. This coincides with our I6 (simulation as sandbox) and I3 (the verifier is the bottleneck); the author phrases it as "Robot RSI still lacks a virtual world"and sees world models as a more scalable answer than classical simulators.

6.3 The Xiaohongshu companion note

The same argument also appears as a Xiaohongshu note, "GPT-6 Astra 开启 Robot RSI 时代! howto 实现"(account ♥VLA 和 RL 的具身未来, 2026-09-06, five text cards tagged # 具身纪元). It adds no papers beyond ASPIRE, RoboClaw and ENPIRE, but it compresses Robot RSI into a quotable definition: "The robot executes the task first. After a failure, the system decides what went wrong, the agent edits code and policy, and the result is verified in simulation and on hardware. Useful experience carries into the next round."That sentence is the natural-language form of the deck's formula $z_{t+1} = A(z_t, \tau_t, r_t, \log_t)$: execution produces τ, deciding what went wrong produces r and log, editing code and policy is A updating z, verification is the selection gate, and carrying experience forward is the recursion. Its closing line, "GPT-6 may not become the robot's brain first; it is more likely to become the engine that manufactures the next generation of robot capability", is the essay's core claim in one sentence.

6.4 Assessment

The essay's central claim, that a frontier LLM is more likely to become the cognitive hub of Robot RSI than the robot's end-effector controller, is supported by evidence this report has verified: ASPIRE uses Claude Opus 4.6 to read multimodal execution records and repair programs, RoboHarness uses GPT-5.5 to edit orchestration code, ENPIRE has coding agents read literature, propose hypotheses and compare them on hardware, and Zetta keeps updating critics, recovery skills and tools with the VLA frozen. Its industry notes (Anthropic's "When AI builds itself", OpenAI's RSI team, the founding and funding of Recursive Superintelligence, Trajectory and Discovery Loop, Wang Xingxing's "let the physical-AI robot model self-evolve directly"at the 2026 World Robot Conference) are observations, not evidence, and the Robocurve test and the washing-machine video are third-party reports. What the essay leaves out is the same as the Xiaohongshu essay: governance and safety. Its closed-loop cell has a definition but no mechanism, while Runtime Governance's 96.2% interception and EmbodiedGovBench's seven dimensions are what would make a closed loop authorisable. A second gap is Lil'Log's rule that the evaluator must stay outside the evolution loop: when the self-evaluation stage is itself being improved (Meta-Rewarding's judge of judges), how that rule is enforced on a robot is the real hard problem of Robot RSI.

7. Trends and insights

Ten judgements from reading the four materials and 154 papers together, each with its evidence, its limits and what it implies for choosing research problems.

I1 The optimised object is moving up the stack: from actions to artifacts

In 2022 an LLM emitted a policy program. In 2026 coding agents optimise skill.md (SkillOpt), multi-file policy repositories (RHO), skill libraries (ASPIRE, PRACTICE), training recipes and infrastructure (ENPIRE, HARBOR), runtime critics and recovery skills (Zetta), and whole harnesses (SHAPER, HarnessOpt). Lil'Log's ladder, prompt → context → workflow → harness code → optimizer code, has been walked end to end on the robotics side. Implication: a new method's contribution increasingly depends on which level of artifact it turns into a searchable, verifiable, reusable object, not on a few points on a benchmark.

I2 “Frozen VLA plus learning around it”is the default, with a ceiling

Harness VLA, BATON, AGM, HyMeS, Zetta, SHAPER, RoboHarness, AtomBridge and RL²-VLA leave the VLA’s weights alone. The reasons are practical: weight updates are expensive, slow, hard to verify, and closed frontier models expose no weights. The results are real (+38.6 points for Harness VLA on LIBERO-Pro, 90.8% for Zetta). So is the ceiling: the essay’s USB example (the continuous relation from vision to contact to force to micro-motion) and BATON’s diagnosis that VLA primitives have no entry condition are holes external learning cannot fill. LWD (95% with 16 robots), Q-Planning (40% → 90% on hardware), TEMPO and Temporal GRPO show that real feedback eventually goes back into weights. HyMeS’s “skills in weights, memory in code”is a workable division: learn low-level motor skills by gradient, learn high-level memory and process management with a coding agent.

I3 The verifier is the new bottleneck and a new scaling axis

Every step of a self-evolution loop asks “did it succeed?”PhyAgentOS separates execution termination from semantic completion; Thea makes evaluation an exit code; AGM advances progress only on physical evidence and concludes that reliable memory rests on update discipline; LLM-as-a-Verifier shows verification accuracy scales along score granularity, repeated evaluation and criteria decomposition and can serve as dense RL reward; VASO replaces trace-level evidence with formal verification; the position paper argues success rate alone cannot certify physical reasoning. Lil’Log’s first RSI challenge is exactly the weak evaluator. PRIMO R1 (a process-level video critic that judges progress and failure location) and VERITAS (an inference-time visual verifier whose verified rollouts become fine-tuning data), both named by the 具身纪元 essay, show verifiers splitting into process supervision and action selection. Implication: in Agent × Robot a better verifier is scarcer than a better planner and more likely to stand as an independent contribution.

I4 Memory splits into in-weight and out-of-weight routes with different domains

In-weight: NativeMEM (32.4% → 84.0%), LaMem-VLA, Remember Smarter, VQ-Memory, SkillMemo, PonderPounce’s native causal context. Out-of-weight: AGM, HyMeS, BATON, concept-centric memory, OnEvoMemory, RoboHarness’s execution memory. RoboMME’s finding (representation effectiveness is task-dependent) and RoboMME-Interference’s finding (perceptual memory decays under distraction; retrieval restores it) give a selection rule: precise timing and counting favour in-weight memory; cross-session, distraction-robust, inspectable and editable needs favour structured out-of-weight memory. The essay’s four memory types fit on both routes.

I5 Harness turned from a software noun into a real-time-systems problem

A software harness intervenes at tool-call boundaries; a robot harness must intervene in control, compute and communication at once and must know worst-case latency, deadlines and fallbacks. PhyAI’s control-time Roofline, EcoVLA’s 20 Hz constraint, CloudEdgeVLA’s 40-step delay, ARLI’s broken Markov assumption and UniFS/Latent Bridge’s frequency stratification are concrete forms of this. Implication: robot-harness research will look more like operating systems and real-time scheduling than prompt engineering; Lil’Log’s analogy between harnesses and operating systems is literal on the robot side.

I6 The cost of physical trial and error makes simulation a sandbox

A software agent retries a thousand times; a robot may break a part on the first try. TwinRL, RoboSnap, Agentic Real2Sim, PerceptTwin, SafeDojo and ContactGuard share one idea: simulation and world models rehearse and verify new skills and plans rather than serve as the main training ground. Each step of the essay’s pipeline (generate skill → static check → digital-twin rollout → domain randomisation → failure mining → safety verification → hardware-in-the-loop → small-scale deployment → real feedback → update the simulation) now has a corresponding tool. Implication: “Real → Sim → Learn → Verify → Real”is an engineerable pipeline; what is missing is the harness that strings the tools together.

I7 Governance and safety became first-class citizens

Runtime Governance (96.2% interception, continuous runtime detection), EmbodiedGovBench (seven governance dimensions), ICAN-Deploy (identity-stable upgrades), FSAR (fleet-level governance), EMBGuard (small guardrail models), SENTINEL and VASO (formal methods), Same Weights Different Robot (action-space metadata), multi-robot communication attacks and AgentRob's hijacking risk together say that a robot that works for a long time must first be governable. Of the four source materials, only ROSClaw and PhyAgentOS touch this. Implication: safety and governance are underrated by the essays and, for now, overrated by the papers, because most of the evidence is still simulated.

I8 Fleets are the route to experience at scale, with sublinear returns

LWD drives one VLA to 95% with 16 robots; ENPIRE cuts convergence to a third or a half with 8 stations, and reports honestly that 8× robots yield 2-3× speed because agents re-explore similar ideas, wait for training, read other branches and analyse logs. What a fleet multiplies is hypothesis throughput. Implication: a fleet's value lies in diversity (different hypotheses, different scenes), not in rollout count for one policy; Lil'Log's "diversity collapse"appears in robot fleets as several agents trying the same idea.

I9 The coding agent becomes the roboticist: autoresearch enters the physical world

RHO's title ("Your Coding Agent is Secretly a Roboticist"), ENPIRE's "physical autoresearch", HARBOR's RL-engineering automation, Nautilus's one-prompt workflows, Goldberg's Push-T experiment (100% with no demonstrations and 46% fewer steps) and AgenticRobotics"you don't need to stay in the loop"are the physical-world version of the AI Scientist, AlphaEvolve and Darwin Gödel Machine line Lil'Log discusses; the 具身纪元 essay names it Robot RSI and traces it back to Eureka and DrEureka's LLM-written rewards and simulation parameters. The physical world adds three constraints: state is hard to read, outcomes are hard to judge, and time matters. Implication: the six failure modes of autonomous research catalogued by Trehan and Chopra (training-data defaults, implementation drift, memory degradation, over-optimism, insufficient domain intelligence, weak scientific taste) will appear in robotics as "100% in sim, 60% on hardware", repeated hypotheses, and declaring success on noise, and need dedicated detectors.

I10 Risks and counter-examples

Three warnings belong next to the nine insights. First, "harness updating is not harness benefit": the ability to write harness edits is roughly flat from 9B to Opus while the ability to benefit is non-monotonic, so harness gains cannot be discussed apart from the base model. Second, reward hacking and Goodhart: Lil'Log insists the evaluator and permission control stay outside the evolution loop, AHE enforces this with read-only verifiers and configuration, and the robot-side equivalents are Runtime Governance's externalised governance and AgenticRobotics'signed verifier; any self-evolving system that puts the verifier inside the editable surface deserves suspicion. Third, dialogue can lower task success in multi-agent systems (World-Model Alignment Through Dialogue), agents rarely call each other as tools (7.09% in Tool-RoCo), and shared experience shares contamination (communication attacks). System complexity is itself a risk in Agent × Robot.

8. Open problems and research suggestions

1. **Scalable verifiers.** Can LLM-as-a-Verifier's continuous scores, AGM's physical-evidence head and VASO's formal checks be combined into a layered verifier and evaluated with EmbodiedGovBench-style metrics? This sets the ceiling of every self-evolution loop.
2. **Entry conditions.** BATON shows VLA primitives have only exit conditions; RoboHarness estimates in-distribution regions from execution memory and steers into them. Can each primitive learn an explicit entry condition and a handoff-quality value (Foresight Residual RL's foresight value) so that long-horizon cost drops from T^K to $T \cdot K$?

3. **Dividing memory between weights and code.** HyMeS's split is a hypothesis. Which information belongs in weights (continuous contact relations) and which in external structure (cross-session, editable)? RoboMME's task taxonomy is a ready-made test.
4. **Real-time-aware harnesses.** Implement Projection / Isolation / Transfer as a ROS 2 Harness Profile connected to MHS, so latency budgets, deadlines and fallbacks become declarative configuration; PhyAI's control-time Roofline is the measurement.
5. **Fleet diversity.** ENPIRE's sublinear speedup comes from repeated hypotheses. Can ShinkaEvolve-style novelty rejection or SkillGLoW-style procedural deduplication make N robots explore N different hypotheses?
6. **Pipelining Real → Sim → Verify → Real.** RoboSnap, Agentic Real2Sim, PerceptTwin and SafeDojo each cover one stage; who builds the harness that runs them per skill?
7. **Governance on hardware.** Runtime Governance and EmbodiedGovBench are mostly validated in simulation; moving the seven governance metrics to real fleets of LWD or ENPIRE scale is the next step.
8. **Measuring Experience Scaling.** The essay's slogan needs metrics. Habilis-β's Tasks per Hour × Mean Time Between Intervention, ENPIRE's MRU/MTU and AgenticRobotics'false-promotion rate are candidates.

A method-story starting point, following slide 24 of the deck: choose a z (for instance, entry conditions and a verification head for skills), choose a source of r (physical evidence plus formal checks), draw the editable surface of A with the verifier outside it, and report the curve of library size versus zero-shot success on a long-horizon benchmark such as LIBERO-Pro Long or RoboMemArena.

9. Appendix

9.1 Paper index

The 208 entries organised by 25 topic lines are in the repository `README.md`; the machine-readable version is `data/papers.csv` (fields: id, short_name, title, authors, year, date, venue, url, code_url, topics, tier, note_zh). The 46 `core` entries are the robotics works named by the five materials, the 94 `extended` entries are 2025-2026 work found in the sweep, and the 68 `foundation` entries are software-side agent, harness and RSI work (including the full reference lists of both Lil'Log posts).

9.2 Glossary

Term	Meaning in this report
Harness	The system around a frozen model that orchestrates execution, calls tools, manages context and state, stores artifacts and evaluates results (Lil'Log's definition)
Code-as-Policy (CaP)	An LLM-generated executable program that processes perception outputs, calls control primitives and may contain feedback loops, used as the policy
Heuristic Learning / System	Deck definition: a coding agent editing software structure (policy, detectors, tests, configuration, memory) from feedback / the object maintained over time
External artifact z	The optimised object in the deck's formula: skill.md, policy.py, skill library, training recipe, git branch, harness code
L1-L7	The essay's self-evolution ladder: Retry, Replan, Reflection, Memory Evolution, Skill Evolution, Policy Evolution, Fleet Evolution
Execution / Learning / Fleet Loop	The essay's three loops: can this task be finished / can the next one go better / can one robot's lesson become the fleet's
Real-time-aware harness	A harness that knows the model's worst-case latency, each skill's deadline, the fallback when the network drops, and which actions must stay local
Projection / Isolation / Transfer	The three enforcement functions of Harness Engineering for Physical AI: gate outputs, bound execution and transmission slots, fall back to a verified baseline
Experience Scaling	The essay's claim that robot scaling comes from continuously generated physical experience across a fleet, not from model size alone
Two axes of RSI	The 具身纪元 essay's frame: what is improved (deployment-time self-evolution / training-time self-iteration / self-evaluation / auto research) × human involvement (in-the-loop / on-the-loop / closed loop)

9.3 Reproduction

- Regenerate the README: `python3 src/build_papers_csv.py && python3 src/generator.py` (the first needs `data/paper_meta.json`, fetched from the arXiv API by `src/fetch_arxiv_meta.py`).
- Download all paper PDFs: `bash scripts/download_papers.sh`.
- Translate with SuperTranslate: set `DEEPSEEK_API_KEY` (or any OpenAI-compatible endpoint) and run `bash scripts/translate_papers.sh`; without a key, `scripts/manual_translate.py` provides the deterministic path (export blocks → hand-filled translation table → in-place rendering → inspect QA) used to produce the six Chinese PDFs in this repository.
- Build the PDF reports and slides: `bash scripts/build_docs.sh` (pandoc + XeLaTeX; the HTML deck is exported to PDF with Playwright/Chromium; the Beamer deck is compiled with XeLaTeX).

9.4 Sources

- 具身 RL 日记 (Embodied RL Diary), "Harness 之后，Agent+Robot 下一站是什么？", Xiaohongshu, 2026. Transcript in `sources/xiaohongshu_harness_next_transcript.md`.
- `code_policy_self_evolving_agents.pptx`, 25 slides; extracted text and speaker notes in `sources/code_policy_deck_extracted.md`.
- "Agent + Robot 论文检索地图", two-page table; transcribed in `data/topics.csv`.

- Lilian Weng, "LLM Powered Autonomous Agents", Lil'Log, 2023-06-23; "Harness Engineering for Self-Improvement", Lil'Log, 2026-07-04. Text archives in `sources/`.
- Marilyn Liu (具身纪元), "GPT-6 未必能当好机器人的大脑，却可能帮王兴兴加速 RobotRSI", WeChat, August-September 2026 (accessed 2026-09-06). Transcript and paper mapping in `sources/wechat_embodied_era_robot_rsi_transcript.md`.
- ♥VLA 和 RL 的具身未来, "GPT-6 Astra 开启 Robot RSI 时代! howto 实现", Xiaohongshu, 2026-09-06. Transcript in `sources/xiaohongshu_robot_rsi_howto_transcript.md`.